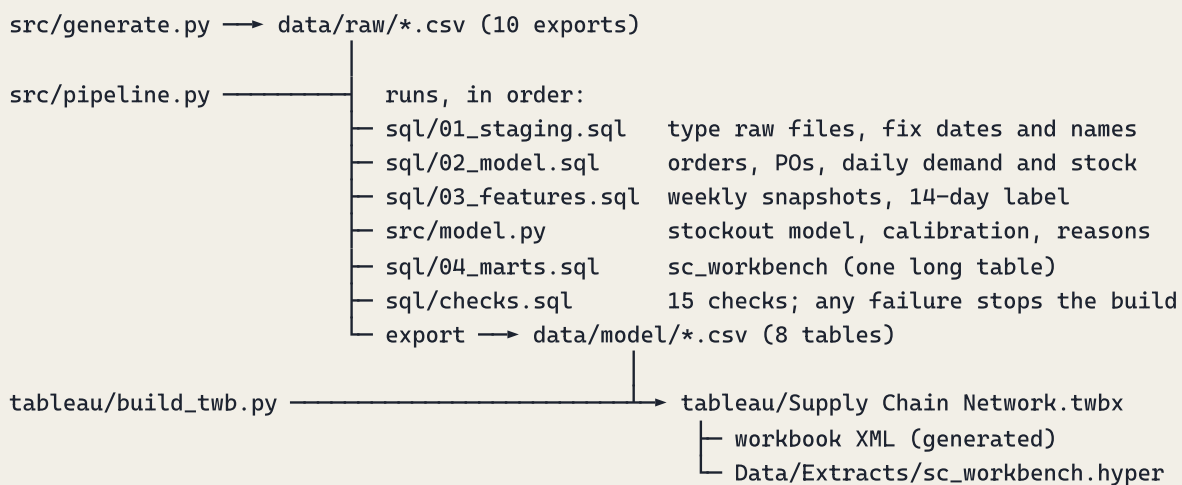


How it works

A walkthrough of the project for reviewers: what each part does, how data moves from raw distributor exports to the Tableau dashboard, how correctness is enforced, and where your eyes are most useful.

Companion to the case study. Paths are relative to the project folder.

1. The flow in one picture



Everything downstream of `generate.py` is deterministic: same seed, same data, same model, same numbers. Two full rebuilds produce identical raw files, model tables and workbook XML; the Hyper extract inside the package is rewritten from the same table on every build.

2. Running it

```

python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
.venv\Scripts\python src\generate.py          # writes data/raw (40 s)
.venv\Scripts\python src\pipeline.py          # builds, trains, checks, exports (60 s)
.venv\Scripts\python tableau\build_twb.py    # writes the .twbx
  
```

Open `tableau\Supply Chain Network.twbx` in Tableau Public 2026.2 (free). The data travels inside the file, so no paths need editing. Tested on Tableau Public 2026.2.2 for Windows.

Expected `pipeline.py` output: three "built" lines, "trained model: Gradient boosting, test ROC AUC 0.840 (reorder-point rule 0.699), stockouts caught at the rule's flag rate 65.6% vs 59.5%", "built 04_marts.sql", 15 lines starting with `pass`, then "exported 8 tables".

3. Stage by stage

3.1 Simulator: `src/generate.py`

Simulates the distributor day by day from 1 April 2023 to 31 March 2026. Each day runs in a fixed order: goods receipts land, retailer orders arrive, warehouses dispatch, replenishment is reviewed, and the end-of-day stock is recorded.

MECHANISM	HOW IT WORKS
Reproducibility	One seeded random generator (<code>SEED = 7</code>). Every product, order, delay and defect comes from it.
Network	4 warehouses; 36 cities, each served by its nearest warehouse. Road distance is great-circle distance times 1.3; the standard transit is road km ÷ 450, rounded up, at least one day.
Suppliers	40 in ten manufacturing hubs, 6 to 10 per category. Each has an on-time probability (Beta(9, 1.6), clipped to 55 to 98%), a defect rate (median about 1.2%, up to 8%) and a short-shipping probability. Quoted lead time: 5 days plus the road distance to the farthest warehouse ÷ 600.
Products	200 in five categories (personal care 50, home care 40, packaged foods 50, beverages 30, snacks 30), with a unit cost, a price 18 to 35% above it, a case size and a popularity.
Demand	Each city orders on two fixed weekdays (Monday to Saturday). An order carries a product with probability p (popular lines almost always), in a Poisson quantity scaled by $1/p$, so demand stays right on average. Volume follows city size, popularity, cheaper products selling more, 8% growth a year, beverages up 45% in April to June, and the Diwali lift (snacks 1.70, beverages 1.50, home care 1.45, personal care 1.35, packaged foods 1.20) ramping up over the 24 days before Diwali and easing over the week after.
Dispatch	Monday to Saturday, first come first served, up to a daily capacity in units: 1.9 times the opening average daily volume, 1.45 at Kolkata, growing 8% a year. Lines ship what is on the shelf; the rest is cut, not back-ordered.
Delivery	Transit days plus a Poisson delay (0.25 a day, plus the city's monsoon exposure from July to September, worse for one of the three carriers), plus a 15% chance of one more day. The promised date is order date + 1 + standard transit.
Replenishment	At the end of each day: reorder point = trailing 28-day average demand × quoted lead time + $1.0 \times$ demand standard deviation × $\sqrt{\text{lead time}}$. When stock plus open orders falls to it, order up to the reorder point plus 30 days, in whole cases. From 1 September 2025 the reorder point is multiplied by the highest Diwali lift expected over the next three weeks (the pre-build).
Purchase orders	Late with probability $1 - \text{on-time}$, by $1 + \text{Gamma}(2, 2.5)$ days; the monsoon adds a Poisson(1.2). Supplier S036 is 93% on time until June 2024 and 45% after. A short shipment delivers 50 to 90%; 15% of orders arrive in two deliveries 1 to 4 days apart; quality control rejects each unit with the supplier's defect rate.

The exports are shaped like ERP and warehouse extracts: purchase-order dates are day first, deliveries arrive split, and every seventh supplier name is in capitals with trailing spaces.

3.2 Staging: `sql/01_staging.sql`

Reads every CSV as text and casts explicitly. Purchase-order dates are parsed from `dd/mm/yyyy` ; supplier names are trimmed and restored to title case word by word, keeping FMCG as an acronym. No business logic lives here.

3.3 Model: `sql/02_model.sql`

TABLE	BUILT HOW
-------	-----------

<code>dim_date</code>	Every day with its week start (Monday), month and Indian financial year.
<code>dim_product</code>	Products with an ABC class by value shipped over the whole period: A makes the first 80%, B the next 15%, C the rest.
<code>fact_order_lines</code>	One row per order line: on time (delivered by the promised date), in full (shipped = ordered), OTIF (both), lead days (order to delivery) and cost of goods.
<code>fact_purchase_orders</code>	Purchase orders with their receipts summed: on time (last delivery by the promised date), in full (received $\geq$ ordered), actual against quoted lead time, rejected units.
<code>fact_demand_daily</code> , <code>fact_inventory_daily</code>	Every product at every warehouse every day (876,800 rows): units ordered and shipped; stock on hand, reorder point, stock value, carrying cost (value $\times$ 24% $\div$ 365), out of stock (stock at zero) and below reorder point.

3.4 Features and the model: `sql/03_features.sql` , `src/model.py`

STEP	HOW IT WORKS
Snapshots	Every product at every warehouse every Monday from 1 May 2023 (122,400). Label: stock reaches zero on any of the next 14 days. Snapshots without 14 days of follow-up get no label (NULL), not a zero.
Features	Eleven, listed in <code>model_features</code> : days of cover (stock $\div$ 28-day average demand, capped at 120), cover including open purchase orders, stock $\div$ reorder point, demand trend (7-day $\div$ 28-day average), demand variability, the supplier's on-time rate and lead-time spread over deliveries received in the previous 180 days, days to the next Diwali (capped at 60), category, ABC class and warehouse.
Split	Out of time: train on snapshots to 30 September 2024 (60,000), test from 1 April 2025 (40,000). The six months between are unused, and the test year includes the pre-build policy.
Models	Logistic regression (scaled, one-hot) and histogram gradient boosting (depth 4, 300 iterations). Logistic is preferred unless the other beats it by at least 0.01 ROC AUC; gradient boosting won by 0.025.
Comparison	ROC AUC, precision-recall AUC, and stockouts caught when each method flags the same share of snapshots as the reorder-point rule (23.4%).
Outputs	Metrics, calibration by decile, permutation importance, then a refit on every labelled snapshot to score all of them. High = 50% or more, Watch = 20 to 50%, Low below.
Reasons	For each snapshot the model is re-scored with one input reset to a typical value (median cover, cover with orders, supplier on-time and lead-time spread; demand trend 1; 60 days to Diwali). Inputs that cut the risk by more than 2 points are listed, largest first, up to two.

The model's input query sorts its rows, so the gradient-boosting validation split, and every result, is the same on every run.

3.5 Marts: `sql/04_marts.sql`

RECORD TYPE	GRAIN AND CONTENTS
Lane	Warehouse-to-city lane $\times$ category $\times$ month, two rows per lane: point 1 at the warehouse with the measures (lines, OTIF lines, lead days), point 2 at the city with location only.

Node	Warehouse × category × day: stock value, cost of goods shipped, carrying cost. Sums are rounded to four decimals, because parallel float sums otherwise differ in the last digit between builds.
Supplier	Supplier × warehouse × month of last receipt: orders, on time, in full, lead and quoted days, units received and rejected.
Snapshot	Warehouse × product × week: stock, reorder point, cover, risk score, band and reasons.

For the map, every row has a `map_point` (the warehouse, or the city at a lane's far end). Node and Snapshot rows join the start point of one of their warehouse's lanes, so neither map layer meets an empty group, which Tableau would draw as a stray mark.

3.6 Checks: `sql/checks.sql`

Each check is a query that returns the rows breaking a rule. `pipeline.py` runs them all, prints pass or fail, and exits with an error before exporting anything if any fail.

AREA	CHECKS
Staging	Supplier names clean.
Orders	One shipment per order with dates in order; orders go to the warehouse serving their city; shipped quantities between zero and the quantity ordered.
Stock	Never negative; balances day to day (yesterday + accepted receipts – shipments = today) for all 876,800 rows; carrying cost at 24% a year.
Procurement	Receipts never exceed the order and rejects never exceed receipts; every purchase order goes to the product's own supplier; ABC classes split value at 80% and 95%.
Model	Label matches a direct recount; no outcome without 14 days of follow-up; supplier features use only deliveries received before the snapshot; one score in [0, 1] per snapshot.
Tableau table	Lines, cost of goods, carrying cost, purchase orders and snapshots reconcile to the facts; every lane has both points; map anchors belong to their own warehouse; every map row has a point.

`src/mutation_test.py` tests the checks: on a copy of the database it breaks the data the way each check guards against, one at a time, and confirms that check fails. 15 of 15 are caught.

3.7 Tableau generation: `tableau/build_twb.py`

- **Format.** Tableau's current XML (version 18.1), in the form Tableau Public saves. The manifest declares `ParameterAction` and `ParameterActionClearSelection`; without them Tableau refuses the click action (error D2E8DA72).
- **Parameters.** Warehouse, Period, Category (string lists with All) and As-of week (a list of Monday dates). Two calculated fields combine them: *In scope* for every sheet, and *In scope on map*, which ignores Warehouse so the map never filters itself. Snapshot rows follow the week, all others the period.
- **Map.** A dual axis on one latitude field (`(lat + lat)`): pane 1 draws lanes as lines with point order as the path, coloured by an OTIF band computed with `{EXCLUDE [point_order] : ...}` so a line is one colour end to end; pane 2 draws a circle per map point, coloured by the share of products at high risk, cities in grey.
- **Click action.** An `edit-parameter-action` on the map sets Warehouse to the clicked mark's warehouse; clearing the selection sets it back to All.
- **Calculations.** 23 calculated fields: OTIF, turnover ($365 \times \text{cost of goods} \div \text{stock-days}$, valid under any filter), lead time, carrying cost, risk counts and shares, supplier measures, and the lane values in the map tooltips.

- **Test switch.** `--preset Warehouse=Kolkata` (and Period, Category, "As-of week") builds a copy that opens with those values; `tableau/filter_check.py` prints the numbers it should show, computed independently in DuckDB.

3.8 Helper scripts

SCRIPT	PURPOSE
<code>tableau/validate_twb.py</code>	Validates the workbook XML against Tableau's schema. A small set of errors is expected (Tableau's own saved files produce the same kinds); it cannot see the manifest feature switches.
<code>tableau/open_twb.ps1</code>	Opens a workbook in a new Tableau Public window, waits until it is drawn at full size, captures it, reads its log, and closes only that window, including on failure.
<code>tableau/capture_docs.ps1</code>	Case-study images: native-size captures cropped to the 1440 × 900 canvas (<code>crop_dashboard.ps1</code>), retaken when Tableau's editor outlines appear; scratch files removed.
<code>tableau/make_extract.py</code> , <code>extract_tableau_xsd.py</code>	Writes the Hyper extract (telemetry off); copies the schema built into the local Tableau install (never committed).
<code>docs/charts.py</code> , <code>docs/build_pdf.ps1</code>	Draws the calibration chart from the model table; prints the docs to PDF with headless Edge (<code>-Review</code> also renders every page for layout review).

4. Design choices worth challenging

CHOICE	WHY	TRADE-OFF
Synthetic data	Publishable and reproducible; planted effects let the methods be checked against a known answer.	The model can only find what was planted.
OTIF per order line	An order of about 90 lines is complete only 16.6% of the time; per line is how most FMCG distributors report it.	Hides how often a retailer gets a whole order.
Short lines cut, not back-ordered	Common practice in FMCG distribution; keeps the stock balance exact.	Lost sales never return as later demand.
Weekly snapshots, 14-day label	Matches a weekly replenishment review.	A product that runs out on the day of a snapshot is labelled by the next one.
Comparing at the rule's flag rate	Same effort, so the gain is in stockouts reached.	Depends on the 23.4% the rule happens to flag.
One long table	Every parameter reaches every view without blending.	Sparse columns; every calculation must respect record type.
Map never filters itself	The network stays in view, so the next click is always there.	The map doesn't show the selected warehouse's numbers.
Generated workbook	Reproducible, reviewable in a diff, every number tested.	Tableau's format is undocumented; tested on one Tableau version.

5. What to review

1. **Definitions.** Do OTIF (per line), lead time (order to delivery), turnover and carrying cost (24% a year) match how you'd measure them?
2. **Leakage.** Can any snapshot feature see past the snapshot day? Look at the open-order window and the 180-day supplier window.
3. **Model.** Is the out-of-time split and the comparison with the reorder-point rule fair, given the policy change in the test year?
4. **Checks.** Is there a failure mode the 15 checks would miss?
5. **Dashboard.** Can an operations head find the weak warehouse, the late supplier and this week's at-risk products within a few seconds? Is anything misleading?

6. Known limits

- The distributor, its suppliers and retailers are synthetic; prices, margins and freight costs aren't modelled.
- Inbound supplier lanes aren't drawn on the map.
- Stock is end of day; intraday stockouts that recover by evening are invisible.
- The workbook is tested on Tableau Public 2026.2.2 for Windows only.